

NEON ESCAPE

Projeto Avaliativo — Computação Gráfica

Jogo 2D de arcade/sobrevivência

Python 3 + Pygame

Coletar energia, evitar drones e sobreviver.

Conceito e objetivo

- Cidade futurista com identidade visual neon.
- O jogador controla uma unidade de fuga.
- Vitória: 1000 pontos OU 60 segundos.
- Derrota: 0 vidas ou 0 energia.

Regras da partida

- Núcleo verde: +100 pontos e recuperação de energia.
- Núcleo amarelo: +250 pontos.
- Drone vermelho: causa dano e consome uma vida.
- Após dano: 1,2 s de invulnerabilidade.
- Dificuldade aumenta a cada 15 segundos.

Controles e estados

- WASD/setas: movimentação.
- ENTER: iniciar/reiniciar.
- I: instruções.
- ESC: pausa durante a partida.
- M: menu durante a pausa.
- Estados: menu, instruções, jogo, pausa, Game Over e vitória.

Ciclo principal

- Leitura dos eventos.
- Atualização por delta time.
- Atualização de jogador, inimigos, coletáveis e partículas.
- Detecção e resposta das colisões.
- Renderização das camadas e HUD.
- Controle de 60 FPS.

Arquitetura do código

- `main.py` — janela e loop principal.
- `game.py` — estados, regras, progressão, colisões e cena.
- `player.py` — movimento, energia, vidas, pontuação e animação.
- `enemy.py` — drones.
- `collectible.py` — itens.
- `particles.py` — partículas.
- `ui.py` — interface.
- `settings.py` — constantes.

Técnicas de Computação Gráfica

- Primitivas 2D.
- Transformação de escala no jogador.
- Animação baseada em tempo.
- Composição por camadas.
- Paralaxe simples.
- Sistema de partículas.
- Coordenadas e pygame.Rect.
- HUD e composição visual.

Colisões e resposta

- Áreas espaciais representadas por `pygame.Rect`.
- Detecção com `rect.colliderect()`.
- Coleta: soma pontos e altera energia.
- Drone: reduz vida/energia e reposiciona o jogador.
- Invulnerabilidade temporária.
- `clamp_ip()` mantém o jogador na área jogável.

Progressão

- Fase aumenta a cada 15 segundos.
- Até 8 fases.
- Intervalo de surgimento dos inimigos diminui.
- Limite de inimigos aumenta.
- Velocidade dos drones cresce.
- O objetivo central permanece simples.

Interface e experiência

- Menu inicial.
- Tela de instruções.
- HUD com pontos, vidas, fase, tempo e energia.
- Partículas como feedback visual.
- Pausa, vitória e Game Over.
- Identidade visual neon consistente.

Testes e estabilidade

- TESTES.md contém o plano de testes.
- Movimento, limites, coleta e colisões.
- Game Over, vitória, pausa, reinício e encerramento.
- Sintaxe verificada com `python -m compileall`.
- Execução gráfica final deve ser validada no computador da equipe.

Evolução

- Marco 1 — concepção.
- Marco 2 — protótipo jogável.
- Marco 3 — versão intermediária.
- Marco 4 — versão final.
- Inserir somente capturas históricas reais, se disponíveis.

Autoria, fontes e IA

- Projeto organizado para a disciplina.
- Referências: Pygame e repositórios indicados no enunciado.
- Recursos visuais desenhados por código; sem mídia externa incorporada.
- ChatGPT: apoio em concepção, código, revisão, documentação e depuração.
- Resultado deve ser validado e compreendido pela equipe.

Demonstração

- Abrir o jogo.
- Mostrar menu e instruções.
- Iniciar.
- Movimentar e coletar.
- Demonstrar inimigos e colisão.
- Mostrar progressão.
- Pausar.
- Mostrar vitória/Game Over.
- Reiniciar.

Conclusão e checklist

- Jogo 2D funcional em Python + Pygame.
- Estados, controles, colisões, HUD e progressão implementados.
- Múltiplas técnicas gráficas.
- README, requirements, créditos, evolução e testes incluídos.
- Antes da entrega: executar, preencher testes, inserir capturas reais, gravar vídeo e fazer backup.